

LABORATORIUM 4. TWORZENIE APLIKACJI BAZODANOWYCH.

Cel laboratorium:

Praktyczne tworzenie aplikacji bazodanowej.

Zakres tematyczny zajęć:

- Podstawowe polecenia SQL.
- Konfiguracja projektu.
- Tworzenie aplikacji do obsługi prostej bazy danych.

Pytania kontrolne:

1. Co jest potrzebne do obsługi bazy danych w projekcie Java.
2. Jakie znasz polecenia SQL?
3. Czy dane w bazie mogą się powtarzać?
4. Czy znasz jakieś biblioteki/frameworku usprawniające pracę z bazami danych?

Baza danych

Baza danych (ang. database) jest zbiorem danych uporządkowanych zgodnie z określonymi regułami. W przypadku relacyjnych baz danych są to dane w postaci tabeli wraz z relacjami pomiędzy nimi. Przechowywanie dużej ilości danych (powiązanych ze sobą różnymi relacjami) np. w pliku tekstowym nie jest zalecanym podejściem. W tym laboratorium zostanie przedstawiony sposób wykorzystania relacyjnych bazy danych w języku Java, bez wchodzenia w specyfikę języka SQL (ang. Structured Query Language) używanego do tworzenia i modyfikowania baz danych.

Niezbędne podstawy SQL

Język SQL został opracowany w latach 70., a więc ponad 20 lat przed stworzeniem Javy 1.0. Od tego czasu był wprowadzany w wielu projektach komercyjnych i wiązało się to z ciągłymi modyfikacjami.

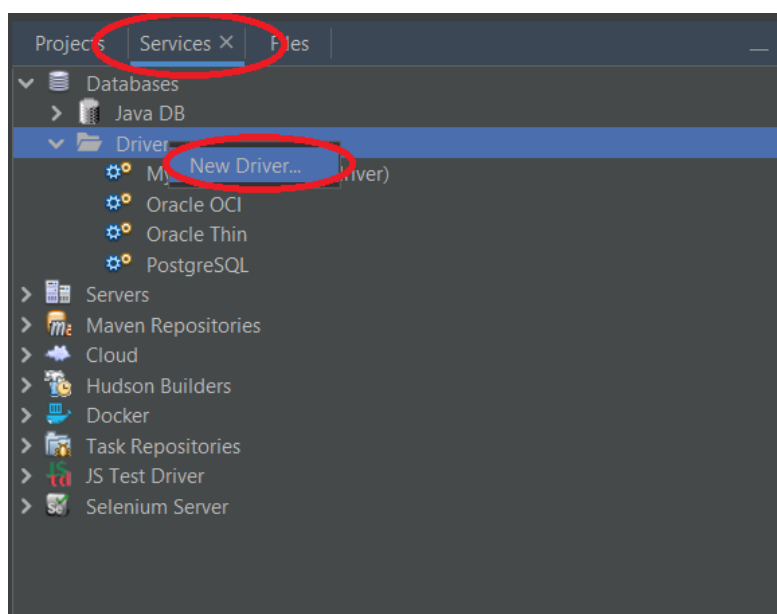
Podstawowymi poleceniami są:

CREATE – tworzy tabelę
DROP – usuwa tabelę
INSERT – umieszcza dane w tabeli
UPDATE – modyfikuje wiersze w tabeli
SELECT – pobiera dane z tabeli
DELETE – usuwa rekord z tabeli

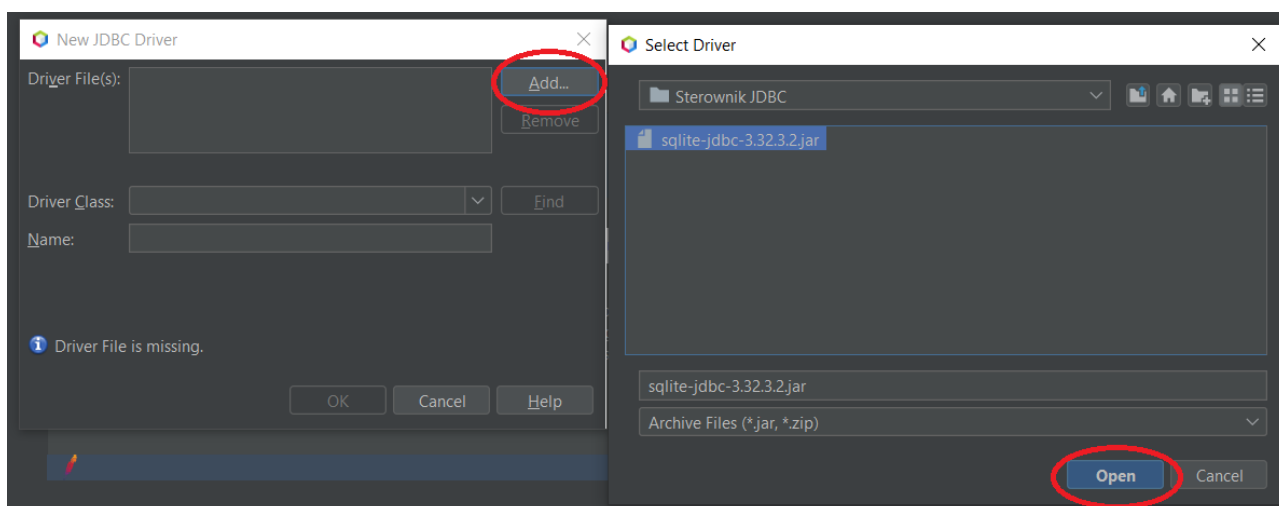
Naturalnie język SQL jest dużo bardziej rozbudowany ale na potrzeby utworzenia i modyfikacji prostej bazy danych przy użyciu oprogramowania napisanego w języku Java powyższe polecenia będą wystarczające.

Konfiguracja projektu

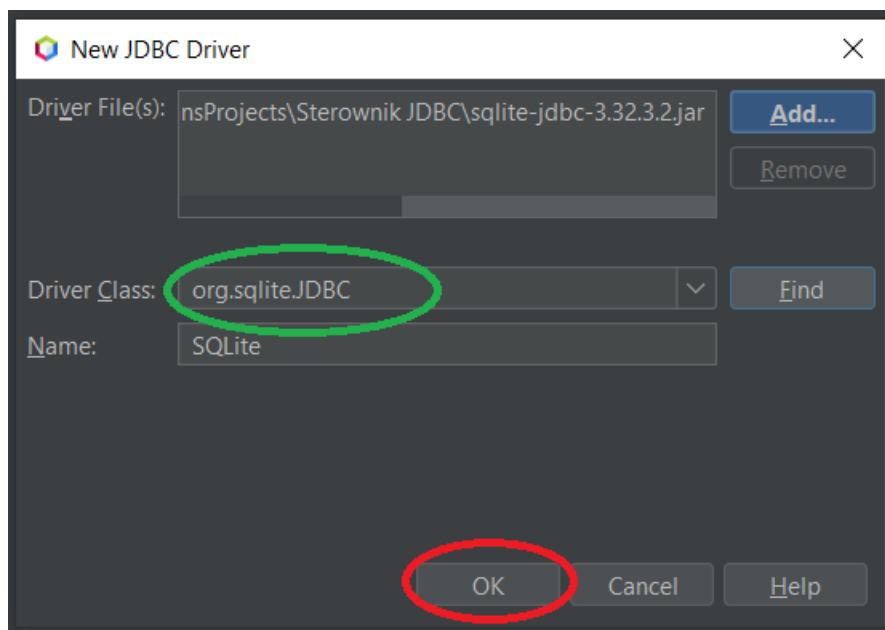
Korzystanie z bazy danych SQL w projekcie Java wymaga użycia specjalnej biblioteki ze sterownikiem JDBC (ang. Java DataBase Connectivity). W zależności od rodzaju bazy danych wymagany jest inny sterownik – dla przykładu zostanie wykorzystany sterownik do bazy danych SQLite. Jest to mała, szybka, wysoce wydajna i niezawodna baza danych (jest to baza bezserwerowa). Format pliku SQLite jest stabilny, wieloplatformowy i wstecznie kompatybilny. Sterownik JDBC można pobrać ze zdalnego repozytorium i dodać ręcznie do projektu lub wykorzystać funkcjonalność Maven'a do zarządzania zależnościami (wówczas należy dodać odpowiednią informację w pliku pom.xml). Dodanie ręczne pokazano na rysunku 4.1. – 3.



Rys.4.1. Dodanie sterownika JDBC – zakładka Services.



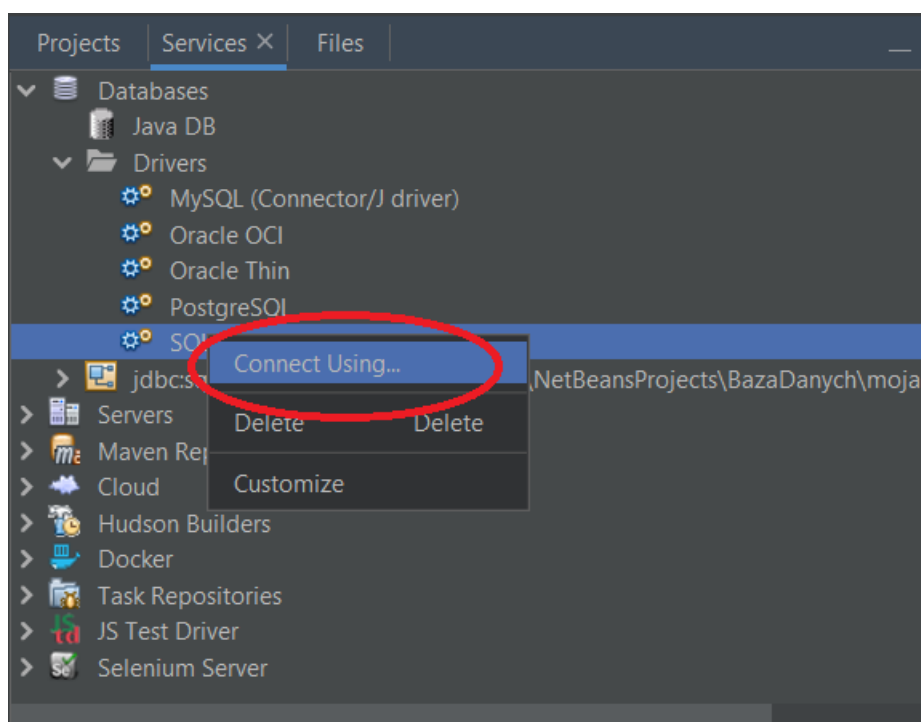
Rys. 4.2. Dodanie pobranego pliku z rozszerzeniem *.jar.



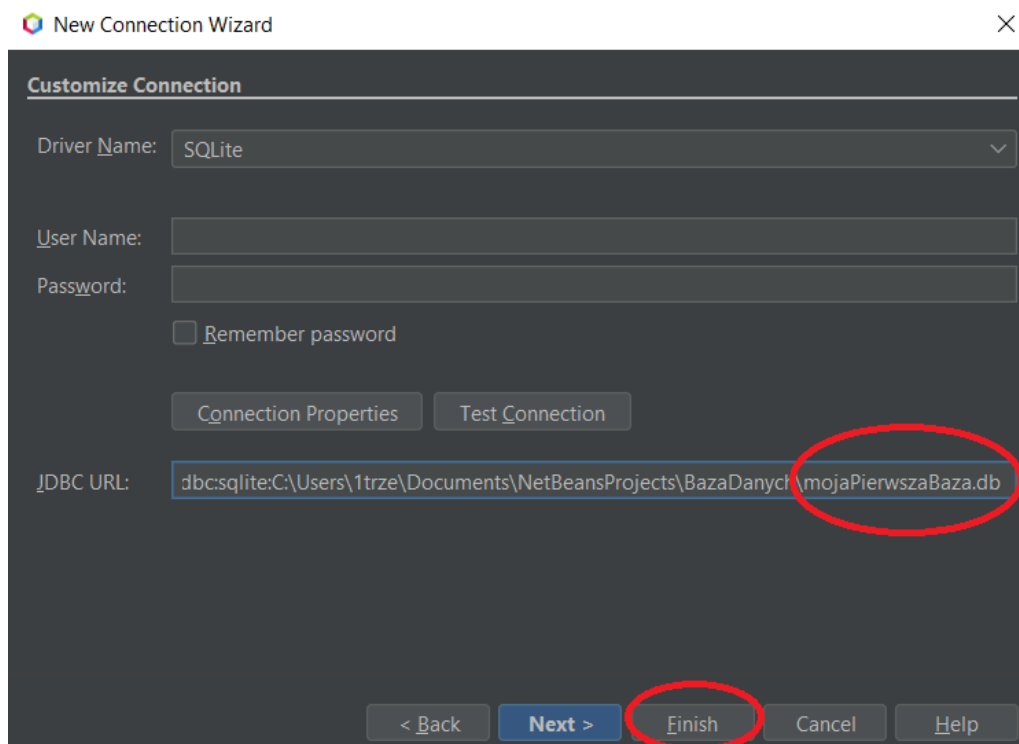
Rys.4.3. Dodany sterownik wraz z zaznaczoną na zielono nazwą klasy.

Po dodaniu sterownika należy utworzyć połączenie klikając PPM na dodany sterownik SQLite → Connect Using...(rys. 4.4), w kolejnym kroku (rys. 4.5) podając ścieżkę dostępu do bazy danych:

jdbc:sqlite:C:\Users\...\NetBeansProjects\BazaDanych\mojaPierwszaBaza.db

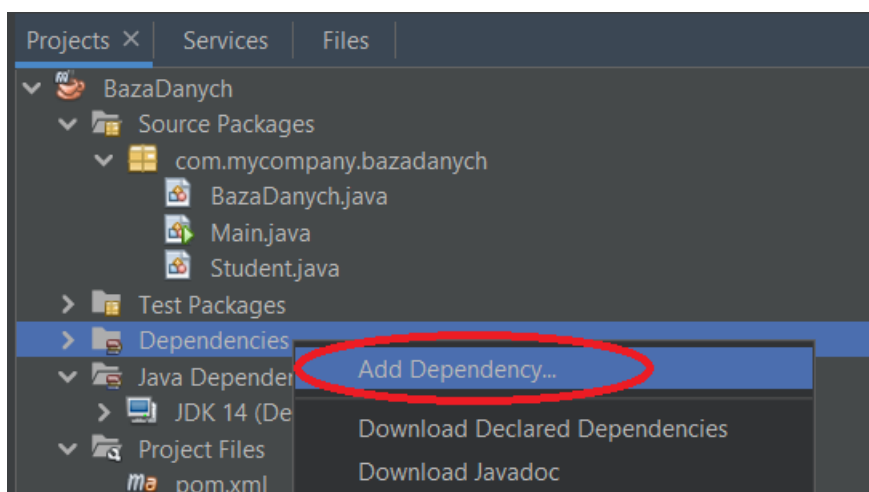


Rys.4.4. Tworzenie połączenia.

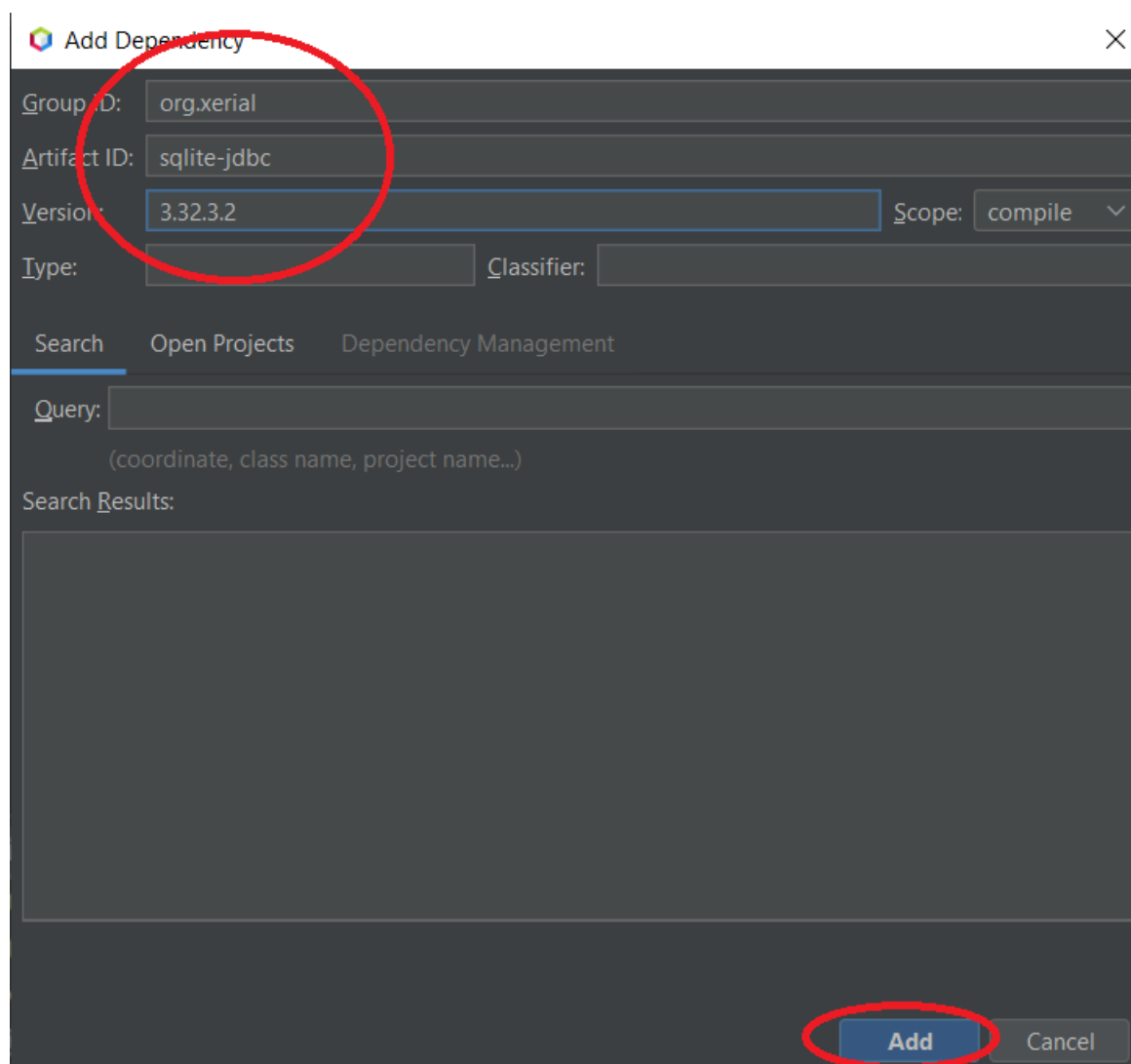


Rys.4.5. Dodanie ścieżki do bazy danych.

Alternatywą dla pobierania i ręcznego dodawania sterownika JDBC jest wykorzystanie Maven'a – w pliku pom.xml należy dodać zależność jak pokazano poniżej (rys. 4.6 – 7). Numer wersji sterownika można sprawdzić w [repozytorium Maven'a](#) – warto korzystać z najnowszej. Po dodaniu zależności sterownik zostanie pobrany.



Rys.4.6. Dodanie zależności – sterownika JDBC.



Rys.4.7. Podanie danych niezbędnych do dodania zależności.

Tworzenie bazy danych

Dla lepszego zapoznania się z tematyką wykorzystywania baz danych w aplikacjach Java poniżej przedstawiono przykład. Projekt składa się z klasy zarządzającej bazą danych (BazaDanych), klasy pomocniczej (Student) oraz klasy głównej (Main). W klasie zarządzającej zdefiniowano cztery metody:

- tworzenie tabeli STUDENCI (tworzTabele())
- wstawianie danych (wstawDane())
- pobieranie danych (pobierzDane())
- zamykanie połączenia (zamknijPolaczenie())

Korzystając z klasy BazaDanych oraz klasy Student możliwe jest utworzenie tabeli STUDENCI oraz dodawanie i odczyt danych z bazy. W metodzie main() zrealizowano przykładowe wstawianie i odczyt danych.

```
package com.mycompany.bazadanych;
```

```
import java.sql.*;
import java.util.LinkedList;
import java.util.List;

public class BazaDanych {

    private static final String DRIVER = "org.sqlite.JDBC";
    private static final String DB_URL =
"jdbc:sqlite:mojaPierwszaBaza.db";

    private Connection connection;
    private Statement statement;

    public BazaDanych() {
        try {
            Class.forName(DRIVER);
        } catch (ClassNotFoundException ex) {
            System.err.println("Brak sterownika JDBC");
            ex.printStackTrace();
        }
        try {
            connection = DriverManager.getConnection(DB_URL);
            statement = connection.createStatement();
            tworzTabele();
        } catch (SQLException ex) {
            System.err.println("Problem z otwarciem
połączenia");
            ex.printStackTrace();
        }
    }

    public boolean tworzTabele() {

        String tworz = "CREATE TABLE IF NOT EXISTS STUDENCI(id
INTEGER PRIMARY KEY AUTOINCREMENT, nazwisko String, imie
String)";
        try {
            statement.execute(tworz);
        } catch (SQLException e) {
            System.err.println("Błąd przy tworzeniu tabeli");
            e.printStackTrace();
            return false;
        }

        return true;
    }

    public boolean wstawDane(String tabela, String nazwisko,
String imie) {
```



```
        try {
            PreparedStatement preparedStatement =
connection.prepareStatement("INSERT INTO " + tabela + " VALUES
(null,?,?)");

            preparedStatement.setString(1, nazwisko);
            preparedStatement.setString(2, imie);

            preparedStatement.execute();

        } catch (SQLException e) {
            System.err.println("Błąd przy wprowadzaniu danych
studenta: " + nazwisko + " " + imie);
            e.printStackTrace();
            return false;
        }

        return true;
    }

    public List<Student> pobierzDane(String tabela) {

        List<Student> wyjscie = new LinkedList<Student>();
        try {
            ResultSet resultSet =
statement.executeQuery("SELECT * FROM " + tabela);
            int id;
            String nazwisko, imie;
            while (resultSet.next()) {
                id = resultSet.getInt("id");
                nazwisko = resultSet.getString("nazwisko");
                imie = resultSet.getString("imie");

                wyjscie.add(new Student(id, nazwisko, imie));
            }
        } catch (SQLException e) {
            System.err.println("Problem z wczytaniem danych z
BD");
            e.printStackTrace();
            return null;
        }
        return wyjscie;
    }

    public void zamknijPolaczenie() {
        try {
            connection.close();
        }
```



```
        } catch (SQLException e) {  
            System.err.println("Problem z zamknięciem  
połączenia");  
            e.printStackTrace();  
        }  
    }  
}
```

```
package com.mycompany.bazadanych;  
  
public class Student {  
  
    private int id;  
    private String nazwisko, imie;  
  
    public Student(int id, String nazwisko, String imie) {  
        this.id = id;  
        this.nazwisko = nazwisko;  
        this.imie = imie;  
    }  
  
    public int getId() {  
        return id;  
    }  
  
    public void setId(int id) {  
        this.id = id;  
    }  
  
    public String getNazwisko() {  
        return nazwisko;  
    }  
  
    public void setNazwisko(String nazwisko) {  
        this.nazwisko = nazwisko;  
    }  
  
    public String getImie() {  
        return imie;  
    }  
  
    public void setImie(String imie) {  
        this.imie = imie;  
    }  
}
```




```
package com.mycompany.bazadanych;

import java.util.List;

public class Main {

    public static void main(String[] args) {

        BazaDanych bazaDanych = new BazaDanych();

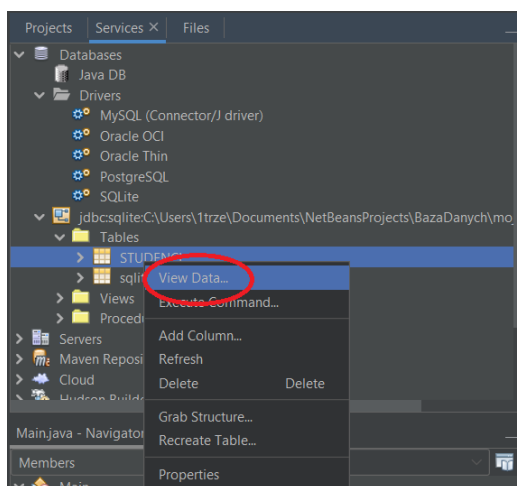
        bazaDanych.wstawDane("STUDENCI", "Kowalski", "Jan");
        bazaDanych.wstawDane("STUDENCI", "Wiśniewski",
"Piotr");
        bazaDanych.wstawDane("STUDENCI", "Nowak", "Michał");

        List<Student> lista =
bazaDanych.pobierzDane("STUDENCI");

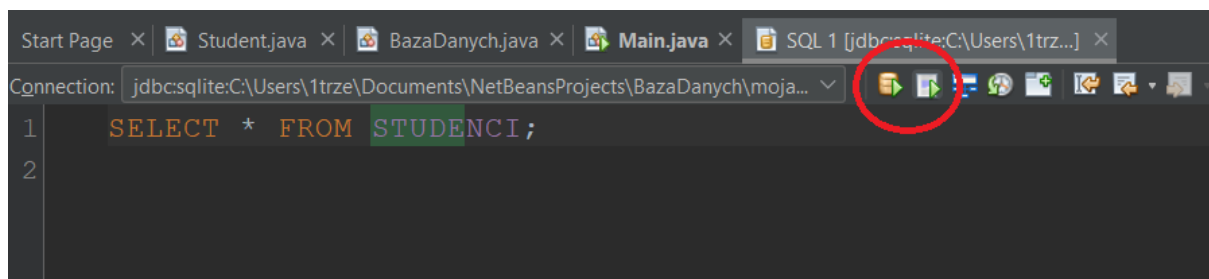
        for (Student s : lista) {
            System.out.println(s.getId() + " " +
s.getNazwisko() + " " + s.getImie());
        }
        bazaDanych.zamknijPolaczenie();
    }
}
```

Po uruchomieniu aplikacji zostanie utworzona instancja klasy BazaDanych, a następnie zostaną wywołane na niej metody: trzykrotnie wstawDane() oraz jednokrotnie pobierz dane. W pętli for – each pobrana lista zostanie wyświetlona.

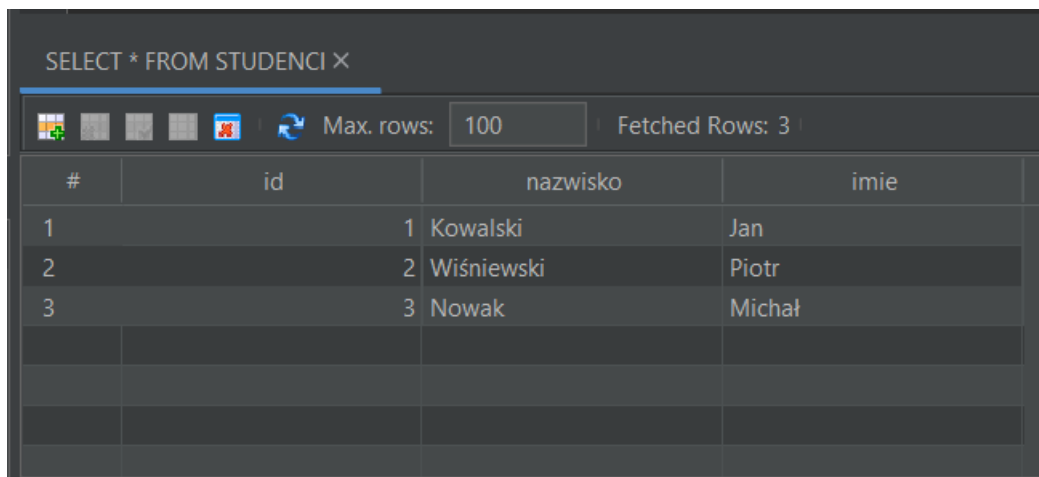
Środowisko NetBeans umożliwia podgląd danych znajdujących się w bazie – w zakładce Services (rys. 4.8) należy wybrać podłączoną bazę i na konkretnej tabeli kliknąć PPM → View Data, a następnie wybrać Run SQL lub Run Statement (rys.4.9). Otrzymane wyniki przedstawiono na rysunku 4.10.



Rys.4.8. Podgląd danych z tabeli.



Rys.4.9. Utworzenie zapytania SQL.



#	id	nazwisko	imie
1	1	Kowalski	Jan
2	2	Wiśniewski	Piotr
3	3	Nowak	Michał

Rys.4.10. Wynik zapytania.

Dane mogą być również widoczne z poziomu dowolnego programu obsługującego bazy danych (np. SQLite Studio).

Zadanie 4.1. Baza danych - rozwinięcie

Wykorzystując kod dostępny w instrukcji dokonaj niezbędnych modyfikacji. Zmieniony kod wykorzystaj do stworzenia aplikacji, która będzie mogła z poziomu konsoli:

- dodawać dane studentów do bazy
- odczytywać dane konkretnych studentów z bazy
- modyfikować dane studentów w bazie (w tym usuwać rekordy)

Zadbaj o obsługę wyjątków, upewnij się, że nie można dodawać danych pustych lub niepoprawnych (podstawowa walidacja), np. nie powinien powstać rekord z wartością w kolumnie imię studenta: Piotr2022. W przypadku pustej bazy danych wyświetl informacje o braku rekordów.

Zadanie 4.2. Baza danych - GUI

Wykorzystaj kod z zadania 13.1 do stworzenia aplikacji GUI. Dodaj niezbędne komponenty (JTextField, JButton, JTextArea...). Aplikacja ma posiadać funkcjonalności wymienione w zadaniu 1.1 oraz przyjemny dla oka graficzny interfejs użytkownika.